计算机系统I 期末复习 Cheatsheet

1. 信息表示与数制

整数表示

表示	范围 / 规则
unsigned n-bit	`0 .. 2^n - 1`
two's complement n-bit	`-2^(n-1) .. 2^(n-1)-1`
sign-magnitude	最高位 sign，其余 magnitude，有 +0/-0
one's complement	负数为正数按位取反，有 +0/-0
biased / excess	`stored = value + bias` ，所以 `value = stored - bias`

补码负数值：若 MSB=1， `value = unsigned_value - 2^n` 。
求 `-x` ：按位取反加 1。
符号扩展：signed 扩展复制符号位；unsigned 扩展补 0。截断：丢高位后按目标格式重新解释。

溢出与乘法

补码加法溢出：

- 同号相加，结果变异号。
- 减法先转成加法： `a - b = a + (-b)` 。
- 硬件判据： `carry into MSB xor carry out of MSB = 1` 。

位宽： `n-bit * m-bit` 结果最多 `n+m` bit。
Booth 1-bit：看 `Q0,Q-1` ， `01` 加 M， `10` 减 M， `00/11` 不变，然后算术右移。

定点小数

小数位权： `2^-1, 2^-2, 2^-3, ...` 。
若有 `f` 个 fractional bits，可以先把整个 bit pattern 当整数解释，再除以 `2^f` 。
定点补码同理：先按补码整数解释，再除以 `2^f` 。

BCD / ASCII

BCD：每个十进制数字用 4 bit，8421 BCD 中 `1010..1111` 非法，可作 don't care。
Excess-3 = BCD + 3。
ASCII：7-bit； `'0'=0x30` ；大小写字母差 `0x20` 。

2. IEEE754 浮点数

类型	总位数	sign	exponent	fraction	bias
single	32	1	8	23	127
double	64	1	11	52	1023

正常数：

```
(-1)^s * 1.fraction * 2^(E - bias)
```

特殊值：

exponent	fraction	含义
0	0	±0
0	nonzero	denormal/subnormal, `(-1)^s * 0.frac * 2^(1-bias)`
all 1	0	±infinity
all 1	nonzero	NaN

十进制转 IEEE：

1. 转成二进制。
2. 规格化成 `1.xxx * 2^e` 。
3. exponent field = `e + bias` 。
4. fraction field 取小数部分，single 23 位，double 52 位。

舍入：默认 round to nearest even。若截断位后是刚好 `1000...` ，看保留部分 LSB，偶数则不进位，奇数则进位。
比较： `+0` 和 `-0` 相等；NaN 不参与正常大小比较；同号正数可按字段大小比较，负数方向相反。

例： `-17.5625 = -10001.1001₂ = -1.00011001₂ * 2^4` 。

- single：sign=1，E= `4+127=131=10000011₂` ，frac= `00011001000000000000000` 。

- double：sign=1，E= `4+1023=1027=10000000011₂` ，frac= `00011001` 后补 0 到 52 位。

3. 组合逻辑 / K-map / Gate Cost

常用布尔律：

```
X + XY = X
X + X'Y = X + Y
X(X + Y) = X
XY + XY' = X
(AB)' = A' + B'
(A+B)' = A'B'
```

K-map：

- minterm 编号仍按 ABCD 二进制算： `0000=m0 ... 1111=m15` 。
- K-map 行/列必须用 Gray code 顺序： `00, 01, 11, 10` ，不是普通二进制顺序。
- SOP 圈 1 和 don't care；POS 圈 0 和 don't care。
- 圈大小为 1/2/4/8/16，越大越好，可重叠。
- 可以跨边界相邻，四角也相邻。
- don't care 只在能扩大圈或降低成本时使用，不必覆盖。

写表达式：

- SOP：圈内不变变量留下，值为 1 写原变量，值为 0 写反变量。
- POS：圈 0，圈内不变变量留下，值为 0 写原变量，值为 1 写反变量。

Prime implicant：不能再扩大的圈。
Essential prime implicant：覆盖了某个只被它覆盖的 1 的 PI。

Gate cost：

- `G` = 非 NOT 门输入数总和。
- `GN = G + NOT 门输入数` 。

NAND/NOR：

- NAND/NOR 是 universal gate。
- SOP 适合 NAND-NAND： `F = (P1' P2' ...)'` 。
- POS 适合 NOR-NOR： `F = ((S1)' + (S2)' + ...)'` 。

4. Decoder / MUX / Verilog

Decoder：

- 3-to-8 decoder 输出 minterm。
- 四变量函数可用两个 3-to-8：一个变量接 enable，另外三个接输入。
- active-high decoder：选中的 minterm 输出 1，最后 OR。
- active-low decoder：输出为 `m_i'` ，选中项可用 NAND 汇总，等价 OR minterms。

MUX：

- 2-to-1： `Y = S ? B : A` 。
- 用 `2^n:1 MUX` 实现 `n+1` 变量函数：n 个变量接 select，剩下变量只会在数据端表现为 `0, 1, X, X'` 。

Verilog：

```
assign Y = cond ? a : b;
always @(*) begin
  ...
end
```

组合逻辑的 `always @(*)` 必须所有分支赋值，否则可能综合出 latch。
数值字面量： `<size>'<base><number>` ，如 `4'b1010` ， `12'habc` 。
`x` unknown， `z` 或 `?` high-Z。负号写在 size 前： `-6'd3` ，不要写 `4'd-2` 。

5. 时序逻辑与 FSM

Latch level-sensitive；flip-flop edge-triggered。

元件	规则
SR latch, NOR	invalid: `S=R=1`
SR latch, NAND	invalid: `S'=R'=0`
DFF	`Q+ = D`
JK FF	`00 hold, 01 reset, 10 set, 11 toggle`
T FF	`T=0 hold, T=1 toggle` ; excitation: `T = Q xor Q+`

Timing：

```
Tclk >= tCQ + tpd_comb_max + tsetup
Fmax = 1 / Tclk
Hold: tCQ_min + tcd_comb_min >= thold
```

FSM：

1. 画 state diagram。
2. 写 state table： `present state, input -> next state, output` 。
3. 状态编码， `m` 个状态至少 `ceil(log2 m)` 位。
4. 用 K-map 求 `D_i = Q_i+` 。
5. 写 output logic。
6. 画 DFF + 组合逻辑电路。

Moore：output=f(state)，较稳定，可能状态多。
Mealy：output=f(state,input)，输出可随输入立即变化，状态可能少，但可能有毛刺。

状态等价：两个状态对每个输入输出相同，且 next state 相同或等价。
未用状态：可作 don't care 优化；若题目要求 self-starting/self-recovery，不能随便作 d，要转回合法状态。
序列检测：状态表示“当前已匹配的最长前缀”；检测后跳到“最长后缀=前缀”的状态，可支持重叠识别。

6. ISA 基础

ISA 是软件与硬件之间的契约，定义指令、寄存器、内存、寻址、异常等。Microarchitecture 是 ISA 的硬件实现。

设计原则：

- Simplicity favors regularity.
- Make the common case fast.
- Smaller is faster.
- Good design demands good compromises.

CISC vs RISC：

CISC	RISC
变长指令	固定长度为主
多寻址方式	少格式、少寻址
复杂译码	简单译码
可有 mem-to-mem	load-store

CISC	RISC
代码密度好	易流水/硬件简单

寻址方式：

方式	含义
immediate	操作数在指令中
register	操作数在寄存器
direct	`EA=A`
indirect	`EA=Mem[A]`
base + displacement	`EA=Reg[base]+disp`
indexed	`EA=A+Reg[index]`
PC-relative	`target=PC+offset`

7. RISC-V 寄存器与调用约定

RISC-V 是 load-store ISA。RV32/RV64 是基本整数 ISA；扩展：M 乘除，A 原子，F 单精度，D 双精度，C compressed/16-bit，G=IMAFD。

Register	ABI	用途	preserved?
x0	zero	常数 0	n/a
x1	ra	return address	no；非叶函数通常入栈保存
x2	sp	stack pointer	yes
x3	gp	global pointer	yes
x4	tp	thread pointer	yes
x5-x7	t0-t2	temporary	no
x8-x9	s0/fp, s1	saved	yes
x10-x17	a0-a7	args / return	no
x18-x27	s2-s11	saved	yes
x28-x31	t3-t6	temporary	no

调用约定：

- 参数放 `a0-a7` 。
- 返回值放 `a0/a1` 。
- `jal x1, func` 调用， `PC+4 -> x1` 。
- `jalr x0, 0(x1)` 返回。
- caller 保存调用后还要用的 `t/a` 。
- 非叶函数或会再次 `jal` 时，要保存 `ra` 。
- callee 保存自己使用的 saved registers。
- stack 向低地址增长：分配 `addi sp,sp,-N` ，释放 `addi sp,sp,N` 。

8. RISC-V 指令格式与 opcode

格式：

Type	Layout
R	`funct7 rs2 rs1 funct3 rd opcode`
I	`imm[11:0] rs1 funct3 rd opcode`
S	`imm[11:5] rs2 rs1 funct3 imm[4:0] opcode`
B	`imm[12,10:5] rs2 rs1 funct3 imm[4:1,11] opcode`
U	`imm[31:12] rd opcode`
J	`imm[20,10:1,11,19:12] rd opcode`

固定字段： `rd[11:7]` ， `rs1[19:15]` ， `rs2[24:20]` ，immediate sign bit 在 `inst[31]` 。
所有 immediate 符号扩展。RISC-V memory byte-addressed；doubleword 相邻地址差 8。

Opcode/funct：

指令	Type	opcode	funct3	funct7/6
add/sub	R	0110011	000	0000000 / 0100000
sll	R	0110011	001	0000000
slt	R	0110011	010	0000000
xor	R	0110011	100	0000000
srl/sra	R	0110011	101	0000000 / 0100000

指令	Type	opcode	funct3	funct7/6
or/and	R	0110011	110 / 111	0000000
lb/lh/lw/ld	I	0000011	000/001/010/011	-
lbu/lhu/lwu	I	0000011	100/101/110	-
addi/slti/xori/ori/andi	I	0010011	000/010/100/110/111	-
slli/srli/srai	I	0010011	001/101/101	000000/000000/010000
jalr	I	1100111	000	-
sb/sh/sw/sd	S	0100011	000/001/010/111	-
beq/bne/blt/bge/bltu/bgeu	B	1100011	000/001/100/101/110/111	-
lui/auipc	U	0110111 / 0010111	-	-
jal	J	1101111	-	-

9. RISC-V 常见汇编格式 / 语义 / 字段

常见汇编格式与语义：

汇编格式	语义
`add rd,rs1,rs2`	`Reg[rd]=Reg[rs1]+Reg[rs2]`
`sub rd,rs1,rs2`	`Reg[rd]=Reg[rs1]-Reg[rs2]`
`and/or/xor rd,rs1,rs2`	按位逻辑运算
`sll/srl/sra rd,rs1,rs2`	移位量取 `rs2` 低位； `sra` 算术右移
`addi rd,rs1,imm`	`Reg[rd]=Reg[rs1]+signext(imm)`
`andi/ori/xori rd,rs1,imm`	I-type 按位逻辑，imm 符号扩展
`slli/srli/srai rd,rs1,shamt`	立即数移位； `srai` 算术右移
`ld/lw rd,imm(rs1)`	`Reg[rd]=Mem[Reg[rs1]+signext(imm)]`
`sd/sw rs2,imm(rs1)`	`Mem[Reg[rs1]+signext(imm)]=Reg[rs2]`

汇编格式	语义
`beq/bne rs1,rs2,L`	条件满足则 `PC=PC+offset`
`blt/bge rs1,rs2,L`	signed 比较分支
`bltu/bgeu rs1,rs2,L`	unsigned 比较分支
`jal rd,L`	`Reg[rd]=PC+4; PC=PC+offset`
`jalr rd,imm(rs1)`	`Reg[rd]=PC+4; PC=(Reg[rs1]+imm)&~1`
`lui rd,imm20`	`Reg[rd]=imm20<<12`
`auipc rd,imm20`	`Reg[rd]=PC+(imm20<<12)`

汇编到机器码字段：

汇编	Type	字段对应	immediate / offset
`add rd,rs1,rs2`	R	`rd,rs1,rs2`	无
`addi rd,rs1,imm`	I	`rd,rs1,imm[11:0]`	12-bit signed
`ld rd,imm(rs1)`	I	`rd,rs1,imm[11:0]`	地址= `Reg[rs1]+imm`
`sd rs2,imm(rs1)`	S	`rs2,rs1,imm split`	store 无 rd，rs2 是写入数据
`beq rs1,rs2,label`	B	`rs1,rs2,offset split`	offset= `label-current PC`
`jal rd,label`	J	`rd,offset split`	offset= `label-current PC`
`jalr rd,imm(rs1)`	I	`rd,rs1,imm[11:0]`	target= `(Reg[rs1]+imm)&~1`
`lui rd,imm20`	U	`rd,imm[31:12]`	`rd=imm20<<12`
`auipc rd,imm20`	U	`rd,imm[31:12]`	`rd=PC+(imm20<<12)`

B/J offset 是 byte offset，但 bit0 隐含为 0，编码时把 offset 按字段重排；基准是当前指令 PC，不是 `PC+4` 。
向后跳 offset 为负，按 immediate 位宽取补码后拆字段。

B-type immediate 拆位：

```
inst[31]=imm[12], inst[30:25]=imm[10:5], inst[11:8]=imm[4:1], inst[7]=imm[11]
branch_target = PC + signext({imm[12],imm[10:5],imm[4:1],0})
```

J-type immediate 拆位：

```
inst[31]=imm[20], inst[30:21]=imm[10:1], inst[20]=imm[11], inst[19:12]=imm[19:12]
jump_target = PC + signext({imm[20],imm[10:1],imm[11],imm[19:12],0})
```

常用伪指令：

Pseudo	Expansion
`li rd,small`	`addi rd,x0,small` ，small 是 12-bit signed
`li rd,big`	`lui rd,upper20; addi rd,rd,lower12`
`mv rd,rs`	`addi rd,rs,0`
`not rd,rs`	`xori rd,rs,-1`
`neg rd,rs`	`sub rd,x0,rs`
`j L`	`jal x0,L`
`ret`	`jalr x0,0(x1)`
`bnez rs,L`	`bne rs,x0,L`
`beqz rs,L`	`beq rs,x0,L`
`sne rd,a,b`	`xor rd,a,b; sltu rd,x0,rd`

宽立即数： `lui rd,upper20; addi rd,rd,lower12` 。若 lower12 的 sign bit=1，upper20 要加 1。
数组越界： `x20 >= x11 or x20 < 0` 可用 `bgeu x20,x11,IndexOutOfBounds` 。

远分支改写：

```
bne rs, x0, L_skip
jal x0, FarLabel
L_skip:
```

10. C 到 RISC-V 常见模板

变量在寄存器中时，C 表达式逐步翻译成三地址形式。数组地址按元素大小缩放： `int` 左移 2， `long long/doubleword` 左移 3。

数组下标：

```
# g = h + A[i]，g=x1,h=x2,i=x4,base(A)=x3，A 为 doubleword
slli x5, x4, 3      # x5 = 8*i
add  x5, x5, x3     # x5 = &A[i]
ld   x6, 0(x5)      # x6 = A[i]
add  x1, x2, x6     # g = h + A[i]
```

if/else：

```
# if (i==j) f=g+h; else f=g-h;  f..j = x19..x23
bne  x22, x23, ELSE
add  x19, x20, x21
beq  x0, x0, EXIT   # unconditional branch
ELSE:
sub  x19, x20, x21
EXIT:
```

while 循环：

```
# while (save[i] == k) i = i + 1;  i=x22,k=x24,base(save)=x25
Loop:
  slli x10, x22, 3
  add  x10, x10, x25
  ld   x9, 0(x10)
  bne  x9, x24, Exit
  addi x22, x22, 1
  beq  x0, x0, Loop
Exit:
```

叶函数：

```
# long long leaf(g,h,i,j){ return (g+h)-(i+j); }
# args x10-x13, return x10；用了 s0/x8 才必须 callee-save
leaf:
  addi sp, sp, -8
  sd   x8, 0(sp)
  add  x5, x10, x11
  add  x6, x12, x13
  sub  x8, x5, x6
  addi x10, x8, 0
  ld   x8, 0(sp)
  addi sp, sp, 8
  jalr x0, 0(x1)
```

递归 / 非叶函数：

```
# long long fact(long long n)，n/return=x10；mul 需要 M extension
fact:
  addi sp, sp, -16
  sd   x1, 8(sp)       # save ra: recursive jal 会覆盖 x1
  sd   x10, 0(sp)      # save n: 递归返回后还要用
  addi x5, x10, -1
  bge  x5, x0, L1      # n >= 1
  addi x10, x0, 1
  addi sp, sp, 16
  jalr x0, 0(x1)
L1:
  addi x10, x10, -1
  jal  x1, fact
  add  x6, x10, x0     # x6 = fact(n-1)
  ld   x10, 0(sp)
  ld   x1, 8(sp)
  addi sp, sp, 16
  mul  x10, x10, x6
  jalr x0, 0(x1)
```

函数调用检查点： `jal` 前保存调用后还要用的 caller-saved 值；被调用函数若改 `s0-s11/sp` 要恢复；递归一定注意 `ra` 和仍需使用的参数。

11. CPU 性能公式

```
ClockRate = 1 / CycleTime
Cycles = IC * CPI
CPUTime = Cycles * CycleTime = IC * CPI / ClockRate
Average CPI = Σ(CPI_i * frequency_i)
Performance = 1 / ExecutionTime
Speedup = T_old / T_new
```

直接求 CPI：

```
CPI = CPUTime * ClockRate / IC
```

单周期 CPU：CPI=1，cycle time 取最长指令路径。
优化后频率： `new ClockRate = new Cycles / new Time` 。
Amdahl：

```
Tnew = Told*(1-F) + Told*F/S
Speedup = 1 / ((1-F)+F/S)
```

指令格式设计题：

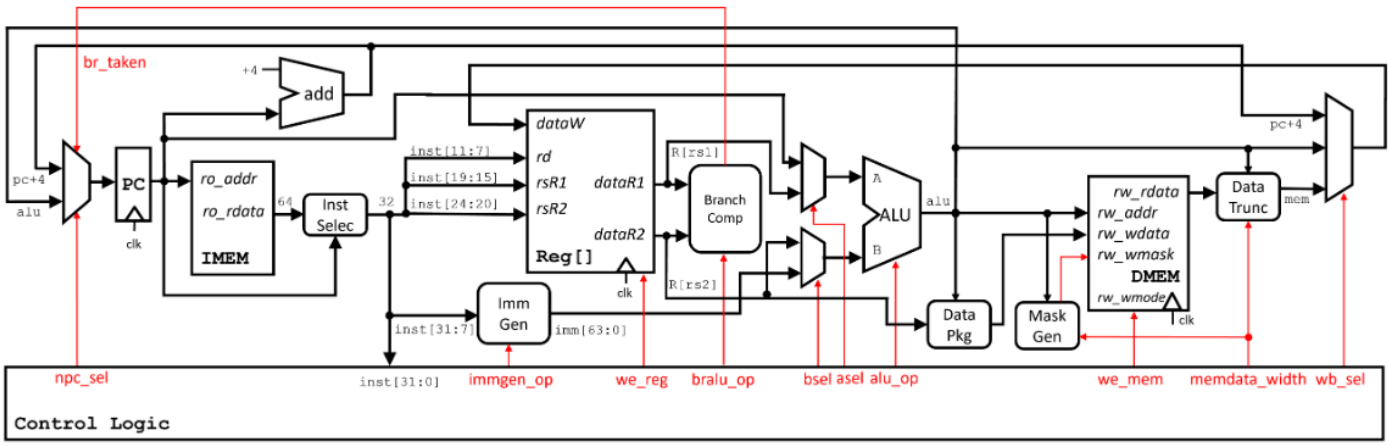
- `N` 条指令 -> opcode bits = `ceil(log2 N)` 。
- `R` 个寄存器 -> register field bits = `ceil(log2 R)` 。
- immediate/address bits =
  `instruction length - opcode - all register fields - other fields` 。
- unsigned k-bit range: `0..2^k-1` 。
- signed 2C k-bit range: `-2^(k-1)..2^(k-1)-1` 。

例： `IC=1e8, CPI=1, cycle=1000ps` ，则 `CPUTime=1e8*1*1000ps=0.1s` 。若新周期数变成 `1.2x` 且总时间变 `0.8x` ，新频率 `=1.2/0.8=1.5x old` 。

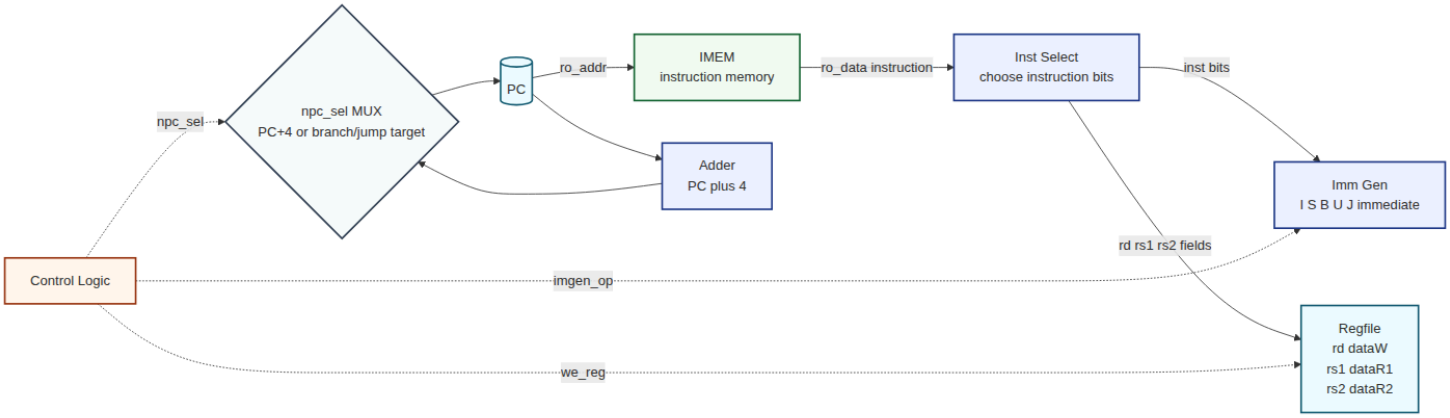
12. Single-Cycle Datapath / Control

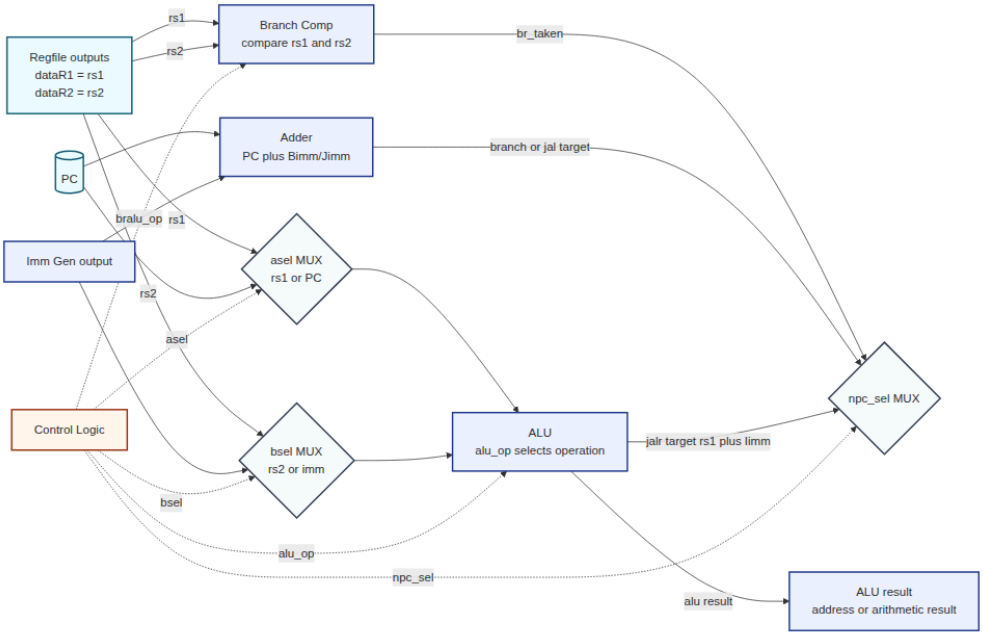
CPU = datapath + control。单周期中每条指令一个 cycle，critical path 决定 cycle time。
Instruction memory 和 data memory 常分开，以避免同周期取指和访存冲突。

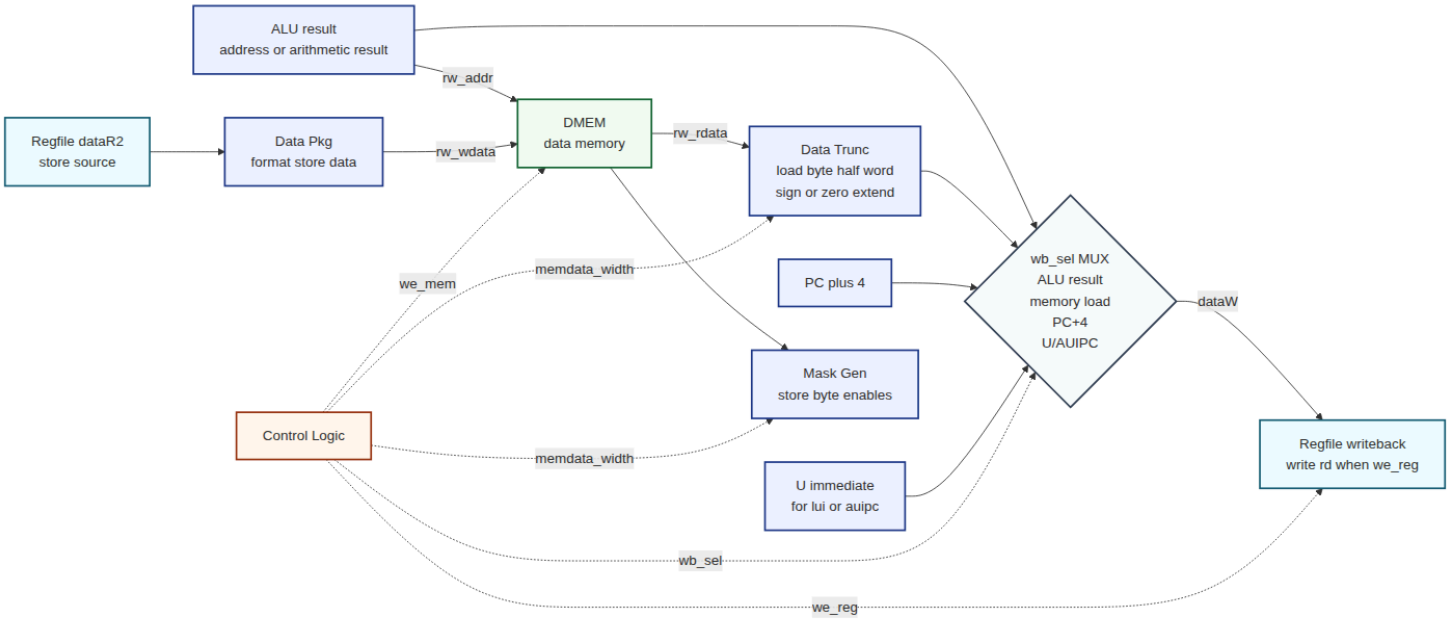
数据通路总图：



分解图：


npc_sel MUX
PC+4 or branch/jump target
npc_sel
PC
ro_addr
IMEM
instruction memory
ro_data instruction
Inst Select
choose instruction bits
inst bits
Adder
PC plus 4
Imm Gen
I S B U J immediate
rd rs1 rs2 fields
Control Logic
imgen_op
we_reg
Regfile
rd dataW
rs1 dataR1
rs2 dataR2


Regfile outputs
dataR1 = rs1
dataR2 = rs2
rs1
rs2
Branch Comp
compare rs1 and rs2
br_taken
Adder
PC plus Bimm/Jimm
branch or jal target
PC
bralu_op
rs1
Imm Gen output
asel MUX
rs1 or PC
rs2
asel
Control Logic
bsel MUX
rs2 or imm
bsel
ALU
alu_op selects operation
jalr target rs1 plus limm
npc_sel MUX
alu_op
npc_sel
alu result
ALU result
address or arithmetic result



基本路径：

指令类	Datapath
R-type	`rs1,rs2 -> ALU -> rd`
I-ALU	`rs1,imm -> ALU -> rd`
Load	`rs1+imm -> DataMem -> rd`
Store	`rs1+imm` 作地址， `rs2 -> DataMem write data`
Branch	`rs1-rs2/compare -> PC select`
Jump	target -> PC, `PC+4 -> rd`
jal	`PC+Jimm -> PC ， PC+4 -> rd`
jalr	`(Reg[rs1]+Iimm)&~1 -> PC ， PC+4 -> rd`
lui	`Uimm -> rd`
auipc	`PC+Uimm -> rd`

控制信号：

Signal	含义
RegWrite	写寄存器

Signal	含义
ALUSrc	ALU 第二输入来自寄存器或 immediate
Branch / BranchN	分支控制，不等于最终 PCSrc
Jump	jump target
MemRead / MemWrite	数据存储器读/写
WBSrc / MemtoReg	`00=ALU`， `01=DataMem`， `10=PC+4`
ALUOp	给 ALU control 的粗粒度操作

图中常见控制名：

Signal	含义
`npc_sel`	选择 next PC： `PC+4 / branch or jal target / jalr target`
`imgen_op`	选择 immediate 格式：I/S/B/U/J
`we_reg`	register file write enable
`bralu_op`	branch comparator 比较类型
`asel`	ALU A 输入选择： `rs1 / PC`
`bsel`	ALU B 输入选择： `rs2 / imm`
`alu_op`	ALU 操作选择
`we_mem`	data memory write enable
`memdata_width`	load/store 宽度与扩展方式
`wb_sel`	写回来源： `ALU / memory / PC+4 / immediate`

典型控制：

Instr	RegWrite	ALUSrc	MemRead	MemWrite	WBSrc	Branch	Jump	ALUOp
R	1	0	0	0	ALU	0	0	R/funct
I-ALU	1	1	0	0	ALU	0	0	I/funct
Load	1	1	1	0	Mem	0	0	add

Instr	RegWrite	ALUSrc	MemRead	MemWrite	WBSrc	Branch	Jump	ALUOp
Store	0	1	0	1	x	0	0	add
beq	0	0	0	0	x	1	0	sub
jal	1	x	0	0	PC+4	0	1	x
jalr	1	1	0	0	PC+4	0	1	add

真正 PC 选择：

- `beq`： `PCSrc = Branch & Zero` 。
- `bne`： `PCSrc = BranchN & ~Zero` 。
- `jal`： `PCNext = PC + Jimm` ； `jalr`： `PCNext = (Reg[rs1] + Iimm) & ~1` 。
- Writeback MUX 至少要支持 `ALU result / DataMem read / PC+4 / Uimm-or-PC+Uimm` 。

ALU control example：

Operation	Control
AND	0000
OR	0001
ADD	0010
SUB	0110
SLT	0111

13. Datapath Extension Template

答题步骤：

1. 写清新指令语义。
2. 列出要读哪些寄存器/存储器。
3. 列出要写哪些状态元件。
4. 看现有 ALU、RegFile read ports、DataMem ports、WB mux 是否够用。
5. 增加必要 MUX/adder/ALU/port/control signal。

常见判断：

- 同周期既要算地址又要算结果，单 ALU 不够，需第二 ALU/adder；若不加硬件则单周期无法完成。
- 一条指令要读 3 个寄存器，2-read-port RegFile 不够。

- Store-like 新指令常改 DataMem write data 来源。
- Load-like 新指令常改 write-back mux。
- `jalr` 需要 `rs1+imm` 作 PC target，并写回 `PC+4` 。
- `lui` 需要 WB source 能选择 upper immediate。